Keywords: multimodal reasoning • auxiliary memory • chain-of-thought • training-free inference

TRAM: Enhancing Multimodal Reasoning with Trajectory-Derived Auxiliary Memory

A Training-Free Inference Wrapper Recovers Reasoning-Derived Information Lost in Long Multimodal Chains Without Any Fine-Tuning

By Kang Liu, Zijing Wang, Yongkang Liu, Mengjie Zhao, Xiaocui Yang, Shi Feng, Yifei Zhang & Daling Wang

arXiv:2608.01922 • Submitted 3 August 2026

As multimodal reasoning chains grow long, intermediate conclusions lose influence over later steps. *TRAM* fixes this without retraining: a training-free inference wrapper extracts a compact trajectory memory from each intermediate step and injects it into decoding, closing the accuracy gap caused by long-chain information loss in Multimodal Large Reasoning Models (MLRMs).

AT A GLANCE

Problem: In long reasoning chains, intermediate conclusions and visual inferences established early in the trajectory lose influence over later decoding steps, causing accuracy to degrade.

Why it matters: Hard visual reasoning benchmarks require chains of many steps; state-of-the-art MLRMs degrade precisely on these long-chain cases where the capability would matter most.

Method: Training-free auxiliary memory extracted from the reasoning trajectory so far and injected as additional context at each decoding step.

Result: Improved accuracy on long-chain multimodal reasoning benchmarks without fine-tuning or model weight modification.

Evidence: Attribution analysis showing correctness correlates with trajectory retention, not visual attribution alone; benchmark improvements across multiple MLRMs.

The Big Picture

Multimodal Large Reasoning Models (MLRMs) combine a vision encoder with a large language model to tackle tasks that require reading an image, constructing multi-step inferences, and arriving at a final answer. Representative models include LLaVA-o1, InternVL-Reasoner, and similar 2026 state-of-the-art systems.

These models work well on short reasoning tasks but degrade on long chains—precisely the cases where step-by-step reasoning is most valuable. TRAM provides a training-free remedy, applicable as a plug-in wrapper around any deployed MLRM.

Why Existing Approaches Fall Short

Retraining with longer context windows is expensive and risks catastrophic forgetting of other capabilities. Models with 32K+ context windows still degrade on long reasoning tasks because the issue is not available context length—the intermediate conclusions *are* in the context—but effective attention to them.

Fine-tuning on curated long-chain examples is data-hungry, hard to generalise, and requires modifying model weights. This prevents deployment on proprietary models accessed via API.

Prompt engineering (e.g., adding explicit "remember that..." prompts) is fragile and task-specific.

TRAM avoids all of these by operating as an inference-time wrapper with no access to model weights.

Core Mechanism

TRAM rests on an attribution insight: correctness in MLRMs is not well-predicted by visual token attribution (how much the model "looks at" the image), but by whether the trajectory up to each step retains and integrates earlier reasoning-derived information. This means the bottleneck is not the visual encoder—it is the propagation of intermediate conclusions through the reasoning chain.

TRAM addresses this by maintaining an explicit **trajectory memory** $\mathcal{M}$ that accumulates reasoning-derived information as generation proceeds:

1. At each completed reasoning step s_t, a lightweight extractor parses the step and identifies key propositions (visual relations, numeric facts, logical constraints, sub-conclusions).
2. These are added to $\mathcal{M}$ indexed by type and step index.
3. When generating the next step s_{t+1}, the most relevant entries from $\mathcal{M}$ are retrieved and prepended to the decoding context as an "Auxiliary Memory" block.

The extractor and retriever are lightweight and rule-based; no neural modules beyond the MLRM itself are required.

Technical Formulation

Let $\mathcal{H}_t = (v, s_1, s_2, \ldots, s_t)$ be the full trajectory at step t, where v is the visual input and each s_i is a reasoning step. The MLRM without TRAM generates:

$$s_{t+1} \sim p_\theta(s_{t+1} \mid \mathcal{H}_t).$$

TRAM augments this with trajectory memory $\mathcal{M}_t$, a structured summary of $\mathcal{H}_t$:

$$s_{t+1} \sim p_\theta(s_{t+1} \mid \mathcal{H}_t, \mathcal{M}_t)\,,$$

where $\mathcal{M}_t$ is constructed by:

$$\mathcal{M}_t = \bigoplus_{i=1}^{t} \text{Extract}(s_i),$$

with Extract parsing the step text and $\bigoplus$ denoting structured accumulation into the memory store.

The attribution score used to motivate the design is:

$$A_{\text{traj}}(s_j, s_{t+1}) = \sum_k \alpha_{k,j}^{(t)},$$

where $\alpha_{k,j}^{(t)}$ is the attention weight from position k in step s_{t+1} to token j in a prior step. The empirical finding is that A_{traj} for early steps decreases as t grows, explaining the long-chain accuracy degradation.

Learning or Inference Procedure

TRAM operates as a post-processing wrapper around any MLRM's generation loop:

1. Load the MLRM (weights unmodified).
2. Initialise $\mathcal{M}_0 = \emptyset$.
3. For each reasoning step t:
   (a) Retrieve top-K entries from $\mathcal{M}_{t-1}$ relevant to the current partial step (keyword + type matching).
   (b) Prepend retrieved entries as an "Auxiliary Memory" block to the MLRM's context.
   (c) Sample s_t from the augmented context.
   (d) Run Extract(s_t) and add results to $\mathcal{M}_t$.
4. Return the final answer from the last reasoning step.

No gradient computation or parameter update occurs at any step.

What the Guarantee Says

The method provides no formal convergence guarantee, but the attribution analysis provides empirical evidence for the causal chain: (i) early trajectory attention decays with chain length; (ii) TRAM injects a compressed representation of the decayed information; (iii) accuracy on long-chain benchmarks improves.

The compression step (Extract) is necessarily lossy, so TRAM provides benefit primarily when the extracted propositions are those the model would have failed to recover from the diluted attention—a condition satisfied empirically for the benchmark tasks studied.

Experimental Findings

Evaluated across multiple MLRMs on long-chain multimodal reasoning benchmarks:

- Consistent accuracy improvement on long-chain reasoning tasks (those requiring > 6 reasoning steps) across all tested MLRMs.
- Minimal or no improvement on short-chain tasks, consistent with the theory that attention dilution is not a bottleneck when chains are short.
- Improvement is larger for tasks requiring explicit recall of visual relations or numeric facts established early in the chain.
- Wall-clock overhead: the extractor and memory retrieval add negligible latency relative to the MLRM's own generation time.

Ablations and Interpretation

Memory type: All three memory entry types (visual relations, numeric facts, logical constraints) contribute; removing any one reduces gains by 15–30%.

Retrieval top-K: Performance peaks at $K = 3$–5; beyond $K = 8$ the auxiliary block becomes too long and begins to hurt attention focus.

Extractor quality: Replacing the rule-based extractor with an LLM-based extractor improves results slightly but increases latency. The rule-based version provides most of the gain at negligible cost.

Baseline—full context: Simply prepending the full $\mathcal{H}_t$ history twice does not help; the structured compression step is essential.

Reference

Kang Liu, Zijing Wang, Yongkang Liu, Mengjie Zhao, Xiaocui Yang, Shi Feng, Yifei Zhang, Daling Wang. **TRAM: Enhancing Multimodal Reasoning with Trajectory-Derived Auxiliary Memory**. arXiv:2608.01922, August 2026. https://arxiv.org/abs/2608.01922